

Intelligent LiDAR Navigation: Leveraging External Information and Semantic Maps with LLM as Copilot*

Fujing Xie¹, Jiajie Zhang¹, Sören Schwertfeger¹

Abstract—Traditional robot navigation systems primarily utilize occupancy grid maps and laser-based sensing technologies, as demonstrated by the popular move_base package in ROS. Unlike robots, humans navigate not only through spatial awareness and physical distances but also by integrating external information, such as elevator maintenance updates from public notification boards and experiential knowledge, like the need for special access through certain doors. With the development of Large Language Models (LLMs), which possesses text understanding and intelligence close to human performance, there is now an opportunity to infuse robot navigation systems with a level of understanding akin to human cognition. In this study, we propose using osmAG (Area Graph in OpenStreetMap textual format), an innovative semantic topometric hierarchical map representation, to bridge the gap between the capabilities of ROS move_base and the contextual understanding offered by LLMs. Our methodology employs LLMs as an actual copilot in robot navigation, enabling the integration of a broader range of informational inputs, while maintaining the robustness of traditional robotic navigation systems. Our code, demo, map, and experiment results can be accessed at <https://github.com/xiexiaoxie/Intelligent-LiDAR-Navigation-LLM-as-Copilot>.

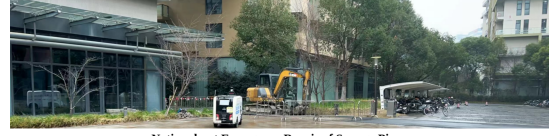
I. INTRODUCTION

In real-world scenarios, robots operating in large dynamic environments face significant challenges, as their surroundings can not always be sensed or updated instantly. An illustrative, real-life example is a campus delivery robot encountering a path obstruction due to an unanticipated pipe repair closure, depicted in Fig. 1. Despite prior notice on public websites, the robot’s navigation algorithm remained unaware of this obstacle. With the capabilities of LLMs, which can process general text-based information, we now have the opportunity to integrate human-like contextual understanding into robotic systems. This way, in the above example, the robot could, by reading the website, from the beginning plan a path that avoids the construction areas, thus achieving a faster and shorter delivery route.

In recent years, the integration of LLMs or VLMs (Vision-Language Models) into robot navigation has opened up diverse avenues for research and application, as highlighted in [1]. Most of these approaches rely on real-time mapping rather than using an a-priori map, as seen in works like [2], [3], [4], often provide only the next immediate step,

*This work has been partially funded by the Shanghai Frontiers Science Center of Human-centered Artificial Intelligence. The experiments of this work were supported by the core facility Platform of Computer Science and Communication, SIST, ShanghaiTech University

¹The authors are with the Key Laboratory of Intelligent Perception and Human-Machine Collaboration – ShanghaiTech University, Ministry of Education, China. {xiejj,zhangjj2023,soerensch}@shanghaitech.edu.cn



Notice about Emergency Repair of Sewage Pipes
on the East Side of the Silk Road Canteen and Road Closures

Dear all,

Due to the emergency repair of the sewage pipe under the road on the east side of the Silk Road Canteen, the road between the east side of the Silk Road Canteen and Student Apartment Building 2 must be excavated and renovated (as marked in red in the picture below). During the repair, the road will be closed for 6 days (January 19 – January 24, 2024).

There will be noise and machinery works. Please stay away and go around the area to avoid accidental injuries.

We apologize for any inconvenience this may cause!

Fig. 1: The figure above depicts a real-life situation encountered by a 3rd-party delivery robot on our University campus, where it is blocked by an intersection closure. Below the e-mail sent by Office of General Services announcing this closure is shown. Reprinted from [6].

lacking long-term planning capabilities. However, without a prior map, LLMs are not naturally aware of a robot’s specific environment. For instance, if asked where to find a pair of scissors, an LLM might suggest searching in a kitchen. When provided with a map of a campus building that includes a robotics lab, the LLM can more accurately direct the robot to the lab. Therefore maps serve as a tool for proper grounding LLMs to the robot’s environment is essential.

For the scenario of this paper we argue, that in robot navigation, we can utilize an existing map that represents the environment and has semantic information. This map, in our case in the osmAG format [5], can be created by a Simultaneous Localization and Mapping (SLAM) approach or from CAD files. Utilizing LiDAR sensors we can localize the robot in this map and employ A* for optimal path planning once the goal coordinates are known. **In our paper we utilize two LLM modules for two distinct tasks: 1) For human-robot-interaction we need to interpret the given language command to identify the navigation goal. For that the LLM leverages the semantic information encoded in the map to find goal coordinates, e.g. by finding the right room for “Please bring this item to the robotics training lab”. This module also utilized experience data to potentially adjust map costs. 2) We interpret external information to adjust map costs.**

When interpreting external information, the LLM needs to match the description of locations (e.g. a specific elevator) with the right entity in the map. This highlights the importance of developing a map representation that is interpretable by both the LLM and laser-based navigation systems, such as move_base, to enable seamless integration of these technologies.

The osmAG map we propose to use in this paper is a hierarchical, topometric semantic map that represents physical spaces as polygon based areas, such as rooms and corridors.

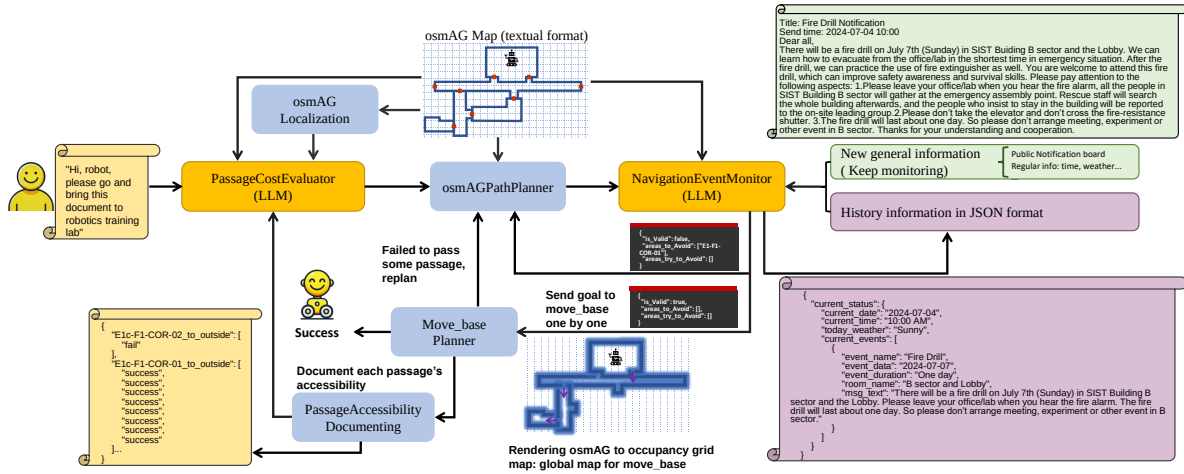


Fig. 2: Pipeline of our algorithm: The *NavigationEventMonitor* keeps tracking events from public notifications and stores information that could impact navigation. When receiving a human instruction, the *PassageCostEvaluator* identifies the destination and assess passage costs based on instructions, accessibility document, and osmAG. This data is used by *osmAGPathPlanner* to plan a path, which is then sent to the *NavigationEventMonitor* for approval. If approved, the path (a list of passages) is sent to *move_base* to move the robot. If not, *NavigationEventMonitor* suggests areas to avoid, prompting *osmAGPathPlanner* to generate an alternative path until approval is granted.

It is compact and textual to ensure compatibility with LLMs for token efficiency and ease of human editing. In [7], we presented a robust and lifelong LiDAR-based localization method that utilize osmAG to find the robot pose by matching the sensor data to the permanent building structures (walls, doors). The map is compatible with traditional robotic algorithms and minimizes update frequency by focusing on permanent structural information.

In this paper, we explore the potential of LLMs serving as co-pilots that understand external information to assist existing laser-based robot navigation systems within large environments. For example, in our experiment, our robot operates in a campus building of approximately 6200 m² and 70+ rooms for one floor. Our work can be quickly employed to real robots with a laser based sensor that are already operating in reality and still benefit from the general knowledge LLM process.

The pipeline of our approach is illustrated in Fig. 2. Our inputs include the osmAG map, human instructions, saved experience files from previous navigation tasks, and tracked events from external notifications. We use real notifications sent by our university administration as the primary source of external information, along with general data such as current time and weather. However, the system isn't limited to just this—humans can easily add other information or preferences into the prompt, allowing the system to adapt to specific needs or situations, e.g. by subscribing to email newsletters from the campus administration and social media channels.

Our experimental results demonstrate that by leveraging LLMs as a copilot, our approach augments traditional laser-based navigation with socially and contextually relevant information, such as public notifications, thereby enhancing robustness and adaptability in human-centric environments.

Our contributions are as follows:

- Our system leverages LLMs to comprehend map data in osmAG format, combining this with historical data and external information sources like public notifications.

- We integrate LLMs into laser-based navigation systems using the same osmAG map, enabling straightforward implementation on real-world robots equipped with LiDARs and an osmAG map, leveraging their ability to plan optimal paths with algorithms like A*.
- We make our code, simulation environments, and experimental results open-source, allowing others to test different LLMs or navigation techniques.

II. RELATED WORKS

A. LLM Guided Robotics

Recent research has integrated robots with LLMs to enhance human interaction and decision making. PaLM-E [8] combines real-world sensor data with language models for improved multimodal reasoning, while [9] fine-tunes PaLM-E using robotic trajectory data to generate actions. Other works utilize LLMs for tasks like object rearrangement [10], creating semantic cost maps for motion planning [11], encoding high-level instructions into actionable trajectories [12], and enabling robots to follow and generate navigation instructions [13]. Additionally, [14] introduces a framework for embodied multi-robot task planning. All these approaches introduce the broad knowledge and reasoning capabilities of LLMs into robotics.

B. LLM Guided Navigation

Most LLM-guided navigation research focuses on using cameras as sensors because visual data makes extracting semantic information easier compared to LiDAR data. For instance, [15] achieves basic navigation by asking an LLM to generate code that calls high-level functions like 'turn_left' to control robot movement. [16] navigates robots based on natural language instructions, such as "After passing a white building, take a right next to a white truck." [17], [18] and [19] build maps using real time sensor data instead of utilizing existing maps. However, these approaches either rely on navigation commands without using proven algorithms like A*, or they treat path planning as part of mission planning,

leaving the LLM unaware of the specific path. In contrast, our paper aims to integrate LLMs directly into the path planning and navigation system, combining the speed and efficiency of A^* with the broad knowledge and intelligence of LLMs for optimal navigation.

C. Semantic maps in Robotics

Armeni et al. [20] and Hughes et al. [21] introduced the Scene Graph, an RGB-D camera-based 3D environment representation that organizes spatial concepts into layered graphs, from detailed semantic meshes to buildings. Later works [22], [23], [24], [25] applied Scene Graphs to language-based localization and task planning. Unlike Scene Graphs, *osmAG* focuses on permanent structures, unaffected by occlusions and changes. Additionally, *osmAG*-based algorithms don't rely on visual data, aligning more closely with traditional navigation algorithms like *move_base*.

III. APPROACH

The *osmAG* map representation is comprehensively defined in [5]. As illustrated in Fig. 3 and the *osmAG* map used in our experiments (Fig. 5), it represents physical spaces as polygon-based areas, such as rooms and corridors. Passages are represented as line segments of the area polygons, connecting neighboring areas (e.g., doors). A formal definition of the *osmAG* is provided in Section III-A.

When navigating familiar buildings, humans rely on experiential knowledge—such as avoiding doors that require special access or choosing automatic doors when their hands are full—as well as external information like elevator maintenance notification. Inspired by this behavior, our system incorporates two key modules to enhance robot navigation. First, as illustrated in Fig. 2, our system employs a module called *PassageCostEvaluator* (described in III-B), which monitors whether the robot successfully passes through specific passages and records this information in a file. This file is then used to adjust cost of passages, enabling the *osmAGPathPlanner* to generate more informed and efficient paths based on past navigation experiences. Second, the system employs the *NavigationEventManager* (described in III-C) to continuously track external notifications. This module saves relevant information into a file, which is used during navigation to assess the validity of the path planned by the *osmAGPathPlanner*. How the *osmAGPathPlanner* plans a path is described in Section III-D, and the integration with ROS *move_base* is in Section III-E.

The use of two files and two LLM-related modules is motivated by the following considerations: When robots navigate large buildings, as shown in Fig. 3, the path is determined by selecting which passages to traverse, as movement between areas requires determinate which passages to pass. However, human communication about locations often refers to general areas rather than specific passages. For example, humans might indicate a fire drill in Sector B of a building, without specifying which doors will be affected. This difference requires to treat passages and areas differently.

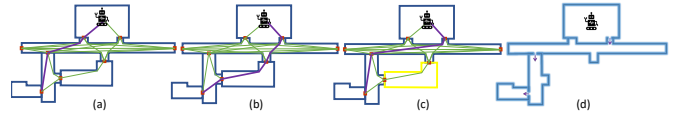


Fig. 3: *osmAGPathPlanning* process: Blue polygons represent areas, red lines denote passages, green paths indicate driving distances between passage pairs within an area (edge cost in *AG*, it is map-specific and stored in a file), and purple paths show the current shortest path. (a) Shortest path based on distance alone; (b) *PassageCostEvaluator* favors the right passage in the start area due to consistently open passages; (c) *NavigationEventManager* advises avoiding the yellow-marked area, likely occupied due to an event; (d) Final path approved by the *NavigationEventManager*, with each passage sent as a goal to *move_base*. The occupancy grid map is rendered as the *move_base* global map, with other passages closed to enforce path adherence.

A. *osmAG* Map Representation

For our navigation task, the topology of the *osmAG* is defined as $AG = (V, E)$, where V represents vertices (passages) and E represents edges (pairs of passages associated with the same area). The cost attached to each edge d_{ij} is the driving distance between a pair of passages. As illustrated by the green paths in Fig. 3, for each *AG*, we use A^* to precalculate the values of d_{ij} based on the polygon of this area. We store these values in a file as they are map-specific and remain constant over time. This file is subsequently used by another A^* algorithm in *osmAGPathPlanner* to compute the optimal sequence of passages from the robot position P_{robot} to the goal area A_{goal} , denoted as $\mathbf{P} = (v_1, v_2, \dots, v_k)$, where $v_1, v_2, \dots, v_k$ are the intermediate vertices (passages).

B. Human Instruction Processing and Passage Cost Evaluation

As illustrated in Fig. 2, initially, the system receives a human instruction, such as ‘Hi, robot, please go and bring this document to the robotics training lab.’ Using the *osmAG* provided in the prompt, the LLM identifies the destination area A_{goal} . In addition to the edge costs d_{ij} described in Section III-A, our approach incorporates an additional cost pc_i for traversing individual passages. This cost pc_i is determined based on two factors: the type of passage (automatic or handle-operated doors) and the robot’s prior experience with it. Some mobile manipulation robots can open doors independently if their hands are free, while others depend on assistance from humans or other robots, or must find alternative routes. As a result, automatic doors may be assigned lower costs compared to doors with handles. Experience data is collected as detailed in Section III-F. Certain doors, such as those frequently blocked by obstacles, are often difficult to open and cause repeated failures, leading the LLM to assign higher costs to these consistently problematic passages.

The prompt to assess pc_i follows the structure illustrated in Fig. 4, includes an overall description, the LLM’s responsibilities, and input/output formats, but without examples or chain-of-thought reasoning. For brevity, the specific prompt is omitted here but is available in our Git repository. The output of this module includes the identified destination area A_{goal} and the pc_i of all passages.

Task: Path Validation and Decision Making
 You are a mobile robot with wheels and also has the ability to take an elevator but not stairs.
 This task involves analyzing the 'current_status' and the provided semantic map to determine if the proposed 'shortest_path' is valid under current conditions. Please don't try to code anything, just analyze the situation using general knowledge like a human who could avoid certain room that is not accessible based on tracked event.

Responsibilities:

1. **Map Interpretation:** When evaluating the shortest_path, only consider the exact areas listed in the path. Do not include areas that are not part of the path, even if they are in the same general sector or building. All the rooms have names that might seem similar, but each name is unique. Pay close attention to the differences between the names to correctly identify each room. Even though the names of the rooms might look alike, the rooms themselves could be located far apart. Therefore, do not assume that rooms with similar names are close or enclose to each other, for example 'E1d-F1-COR-01', 'E1-F1-COR-01', 'E1d-F1-COR-02' are three completely different areas. Each room or area might have tags or labels that give more information about it. These tags help to better understand the specific characteristics or purpose of each room or area.
2. **Decision Making:** First given the 'current_status' and map data, find out which areas are not accessible at this moment, consider them as inaccessible when absolutely necessary and directly affected at the current time. Then based on the inaccessible areas decide if the 'shortest_path' is valid. 'valid' means no area directly in the shortest path is in 'areas_to_avoid'. The shortest path itself already include all areas the robot will pass, so you need to strictly follow the provided shortest_path and do not infer more areas in the shortest_path. If it is valid, set 'is_Valid' to true. If not valid, list the inaccessible rooms in 'areas_to_avoid'. If unsure, use 'areas_try_to_avoid'. Be cautious but not overly restrictive to ensure navigation success. Be aware, areas not directly mentioned in the current_status are default accessible.
3. Based on semantic information from the map and current events, suggest additional rooms to avoid as less preferred paths (areas_try_to_avoid). You don't have to output this only if you think it is necessary.

Inputs:

1. 'shortest_path': A list of rooms representing the shortest path to be evaluated. There maybe similar but not same names, please distinguish them carefully.
2. 'map': The 'map' is provided in OSM (XML) format, where rooms and corridors are represented as areas. The building's hierarchical structure is indicated by the 'parent' tag; otherwise, areas are not nested. Sequential or similar names do not imply inclusion or proximity. Names may be similar but not identical, so distinguish them carefully. Treat each room or area based on the provided facts, avoiding assumptions about their relationships.
3. 'current_status': Events tracked by another program that may affect path planning tasks. One event only affect one area if not saying otherwise.

Outputs:

1. A JSON object indicating whether the current path is valid, a list of areas names to definitely avoid and a list of area names to preferably avoid (areas_try_to_avoid), please only output area name in the map's key-value pair to the list. The map has a hierarchical structure. In your output, please display the leaf areas only, not their parent nodes.

```
{
  "is_Valid": true/false,
  "areas_to_Avoid": ["AreaName1 (not description)", "AreaName2", ...],
  "areas_try_to_Avoid": ["AreaName1", "AreaName2", ...]}

```

Here is your input:

```
{
  "shortest_path": [{"shortest_path"}],
  "current_status": [{"current_status"}],
  "map": [{"map"}]}

```

Fig. 4: The prompt for the *NavigationEventMonitor* module instructs the LLM to assess the validity of a robot's navigation path. The prompt begins with a high-level task description, followed by detailed instructions for the LLM. It then outlines the input parameters and defines the expected output format. Finally, the prompt provides the necessary data for evaluation, including the path being assessed, the tracked current status, and the semantic osmAG map.

C. Navigation Event Tracking and Path Validation

As shown in Fig. 2, the *NavigationEventMonitor* is a second LLM that has two main tasks: It tracks events that may impact the robot's future navigation tasks, such as elevator maintenance notifications. This is done once at the beginning of a navigation task. Additionally, this module validates paths generated by the *osmAGPathPlanner* against tracked events (closed areas).

When information about an external event is received, the LLM determines whether it affects the current or future robot navigation task. If so, the LLM generates a JSON string, which is a human language format summary of this event as well as all previously saved events, which the module stores as event tracking data for later use during path approval. See Fig. 2 for an example. In addition to notifications, the module also tracks general information, such as time, it removes events from tracking once they are no longer relevant.

Additionally, this module validates paths generated by the *osmAGPathPlanner* (Section. III-D) by checking the event tracking data, ensuring the shortest path avoids invalid areas. The module also accounts for the robot's capabilities. For instance, in our multi-floor experiment (Section. V-D), the module recognize that a wheeled robot should use elevators instead of stairs when navigating between floors.

The prompt used in this module to assess the path is shown in Fig. 4. It describes the task and specifies the input and output formats, without examples or chain-of-thought reasoning. The output of this module includes validation of the shortest path and, if the path is invalid, identifies which areas should be avoided and puts those in the 'areas to avoid' field, denoted as A_{invalid} . Since the LLM can infer potential impacts, for instance, if a notification states that main corridors will be closed for classroom renovations, the LLM may infer that other corridors in the same sector might also be affected. An additional field called 'areas try to avoid' denoted as A_{soft} , is included, assigning extra costs d_{soft} to try to avoid these areas without avoiding them completely. Compared to pure A*, this module brings in additional computation time due to LLM inference, each online query taking on average about 2 seconds. However,

by identifying and avoiding invalid or unfavorable areas, it ultimately reduces overall navigation time that would otherwise be spent traversing unsuitable paths.

D. osmAG Path Planner

This module utilizes the precomputed costs between passage pairs d_{ij} , incorporates additional passage costs pc_i , and integrates updates from the *NavigationEventMonitor* if the previously planned path is determined invalid or the intermediate areas of $\mathbf{P}$ are in A_{soft} . The total cost t_{ij} of an edge (v_i, v_j) in AG is given by:

$$t_{ij} = d_{ij} + d_{\text{soft}} + pc_i + pc_j$$

The passages associated with an inaccessible area A_{invalid} are denoted as $E_{\text{invalid}} \subset E$. Removing them from E gives:

$$E' = E \setminus E_{\text{invalid}}$$

To incorporate arbitrary start and goal coordinates into the graph, we need to add the appropriate vertices and edges. For the robot's starting position P_{robot} , we add edges between P_{robot} and all passages in the start area (A_{start}). Similarly, for the final destination, we include the centroid of the destination area and create edges connecting it to all passages within the destination area (A_{goal}). Let v_s represent the robot's starting position P_{robot} , and v_d represent the centroid of the destination area.

The updated set of vertices V' and edges E'' can be expressed as:

$$V' = V \cup \{v_s, v_d\}$$

$$E'' = E' \cup \{(v_s, v_i) \mid v_i \in A_{\text{start}}\} \cup \{(v_d, v_j) \mid v_j \in A_{\text{goal}}\}$$

The final graph incorporating the start position and the destination centroid for this specific navigation task is:

$$AG' = (V', E'')$$

Finally path $\mathbf{P}$ is defined as: $\mathbf{P} = A^*(AG')$, where A^* represents the well-known A star algorithm. Fig. 3 demonstrates our approach on an example: The A^* method selects the path in (a) only utilizing pre-computed d_{ij} . However, with insights from the *PassageCostEvaluator*, the path in (a) is assigned a higher cost, leading to the path in (b).

The *NavigationEventMonitor* then excludes the invalid area (yellow area), resulting in the final approved path in (c).

Utilizing osmAG’s room-level topology as the underlying graph structure, instead of the high-resolution occupancy grid, substantially decreases the computational burden of A* search and allows for more efficient plan and re-plan.

E. Integration osmAGPathPlanner with ROS move_base

After the path $\mathbf{P} = (v_1, v_2, \dots, v_k)$ receives approval from the *NavigationEventMonitor* module, $(v_1, v_2, \dots, v_k)$ will be sent to the move_base actionlib [26] as goals. **Concurrently, the osmAG map is rendered as an occupancy grid map and published as the global map in move_base. This is done by rendering the polygons as occupied cells in the free grid map and then painting the cells of chosen passages as free again. This rendered map is restricted to the chosen areas and the selected passages, ensuring that the move_base global planner strictly adheres to the path provided by the osmAGPathPlanner as illustrated in Fig. 3(d). In move_base, we use the commonly employed ‘Navfn’ as the global planner with the rendered map and ‘DWA’ (Dynamic Window Approach) as the local planner with a local map created from the global grid map updated with LiDAR scans. Despite utilizing documented experiences from previous tasks and external notifications, the robot may still encounter inaccessible passages. In such cases, our method receives a failure message from move_base as the rendered map is restricted to the selected areas and passages, preventing move_base from finding an alternative path. When a passage failure occurs, it is documented, and the osmAGPathPlanner module re-plans the path from current robot position to A_{goal} until the robot successfully reaches A_{goal} .**

F. Documenting Passages Accessibility

During navigation, this module continuously documents the success or failure of each passage traversal. The *PassageCostEvaluator* module sends this stored experience passage data to LLM to evaluate each passage cost pc_i . Furthermore, if a passage is found to be inaccessible during a trial in experiment, the module marks it as ‘infeasible’ and excludes it from the AG’ during re-planning.

IV. EXPERIMENTS

The goal of this study is to integrate laser-based robot navigation using osmAG with external information. A direct comparison with other LLM-aided navigation methods is impractical due to several key differences. **First, our approach relies exclusively on laser sensors, while most existing methods depend on camera-based inputs.** Second, our test environment is significantly larger, focusing on spaces where announcements and notifications are common—unlike smaller, household-scale environments like apartments, where they are rarely used. Additionally, our method integrates real external notifications, which would not make sense in other environments or systems. These fundamental differences in sensor setup and environment scale limit the feasibility of a fair comparison with other existing LLM aided navigation.

Therefore, we conducted our experiments in Gazebo simulation using an actual osmAG map of our campus building, enriched with real notifications from our campus administrative department. This setup allows us to evaluate our system’s performance and compare it with the ROS move_base framework. In Section IV-A, we outline our experiment setup. Section IV-B details how we assess the LLMs’ ability to track navigation-related information and validate a proposed shortest path. Section IV-C compares our method with ROS move_base and Section IV-D evaluates the contributions of the two LLM modules to our approach in an ablation study. Finally, Section IV-E demonstrates our system can operate across floors and avoid elevators under maintenance.

A. Experiments Setup

1) *Robot and World*: In our experiments, we simulate the ‘turtlebot3 waffle’ robot equipped with a 6-meter LiDAR sensor. To ensure a fair comparison, the baseline move_base package shares identical parameters with our approach. We use ROS Gazebo to simulate an environment based on an osmAG of a campus building, covering over 6,200 square meters and 70+ rooms per floor. The baseline move_base is utilizing a rendered osmAG of all areas and passages not occupied. **This occupancy grid map is the global map, which is then updated using the LiDAR data.**

In our experimental setup, we have confined the robot’s operations primarily to the first floor for two main reasons. First, the move_base package, the baseline, lacks multi-floor navigation ability. Second, allowing the robot access to multiple floors could simplify the avoidance strategy by merely shifting its route vertically (moving upstairs or downstairs) to bypass obstructed areas. By confining the tests to the first floor, we can better challenge the algorithm to navigate longer detours outside the building.

2) *Cases*: To evaluate the system’s ability to interpret external information, we designed 5 experimental cases (4 on the first floor, 1 spanning 2 floors), each involving a simulated instruction context and a real event notification. **For example, an instruction might be: “Hi, robot, please bring these tools back to the same room but on second floor.”** This is then combined with event details sent to students by the campus administration, including specific times, events, durations, and locations, as illustrated in Fig. 2. To test our approach to encode experience, in some cases, in the simulator, we close the doors of restricted areas, while in others, we leave them open. Those closed doors cause the according passage to be obstacles in the LiDAR map. The baseline move_base will update the global map accordingly and plan then alternative path. For our method, our move_base will report a failure to reach the goal point (next passage) and we mark that passage as closed and then re-plan the path as described in Section. III-E. In each case, there are designated areas that the robot should consider restricted (shown in Fig. 5), as indicated by prior notifications. Several starting areas are established for each case, along with destination areas, each pair forming one trial. The trials are designed such that the shortest path

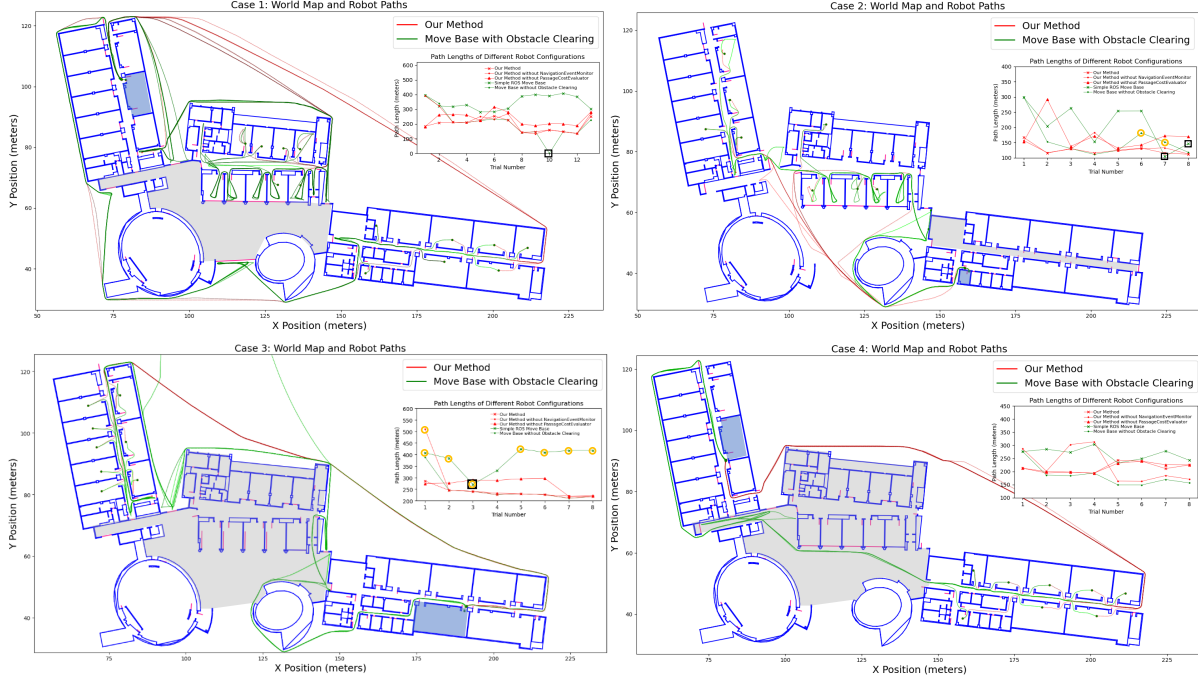


Fig. 5: This figure presents the paths from four experimental cases. For clarity, only the paths from our method (red) and move_base with clearing between trials (green) are displayed on the map. In the map, blue lines represent areas in the osmAG, while red lines indicate passages between them. Each case was tested in a Gazebo world with different door configurations (open or closed). Grey shaded zones represent restricted areas caused by events in case 1): a graduation party in the lobby, 2) classroom renovation in the D sector, 3) a fire drill in the lobby and B sector, and 4) a wireless access upgrade in the lobby and B sector. Blue shaded zones represent destination areas of each case. The inset figures show the path lengths for each trial, comparing our method, our method with ablation testing, move_base with and without obstacle layer clearing. The black box highlights where move_base failed, and the data within the yellow circle shows where the path intruded into restricted areas. In Case 4, only the path with *NavigationEventMonitor* avoids intruding into restricted areas.

typically passes through restricted areas, requiring the robot, when notified, to intelligently plan a detour around them.

Each case uses a specific Gazebo world, and the experience file in our method begins empty for each case. As the system navigates, each passage’s accessibility is documented, gradually building a comprehensive understanding of the environment to optimize performance in subsequent trials.

We defined 4 different cases of area closures through external information and then sampled 37 trials (start and goal area pairs). We then conducted test with our system (1x ours + 2x ablation study) and 2 versions of the baseline, totaling $37 \times 5 = 185 + 1$ (multi-floor) navigation runs in Gazebo, which are explained in detail below.

B. NavigationEventMonitor LLM Experiments

Here we test the first task of the *NavigationEventMonitor* module, which assesses the relevance of each notification. We check the pure LLM performance of correctly identifying if a notification is or may become relevant. To evaluate the module’s ability to filter relevant information, we collected 10 navigation-related notifications and 10 unrelated ones (e.g., water cut-off notifications) and evaluated their correctness by hand. Additionally, after the module successfully tracks the events, we send notifications with a current time that is after the event’s end time. This tests the system’s ability to disregard outdated information. Success is defined as the module correctly identifying relevant notifications and removing event summaries once the event time has passed.

Furthermore, we evaluate the *NavigationEventMonitor* module’s second task: identifying areas to avoid for affected *osmAGPathPlanner* paths and confirming the validity of unaffected paths, based on the string of event summaries that was generated above. We conducted 37 trials across 4 cases, sending a total of 95 path approval requests to ChatGPT-4o, so we re-planned 58 times upon encountering a closed door. In this experiment we also report recall (the proportion of invalid paths correctly identified) and false positive rate (the proportion of valid paths incorrectly flagged as invalid). Alongside ChatGPT-4o, we wanted to also test DeepSeek-V3. However, due to the official API being unstable at the time of writing, evaluation was limited to 40 path approval requests, half of which were valid and the other ones invalid. Thus, for navigation simulations, we only used ChatGPT-4o.

C. Comprehensive Comparison to Baseline

With the above mentioned 37 trials in 4 cases we comprehensively test the whole navigation system and compare our method to the ROS move_base baseline, with and without clearing obstacle layer of the global map of move_base between trials. Note that the results from move_base without obstacle layer clearing closely approximated the actual shortest path in later trials in one case. But in real life implementation, the global map needs periodic clearing to prevent sensor noise, environmental changes, or dynamic objects from being treated as permanent. The performance is evaluated based on the total navigation path length and

TABLE I: Path Validation Comparison: ChatGPT-4o vs. DeepSeek-V3 (%)

Model	Accuracy	Recall
ChatGPT-4o	0.96	1.0
DeepSeek-V3	0.93	0.9

whether the robot enters restricted areas.

D. Ablation Experiment

To evaluate the contribution of individual components in our system, we conducted an ablation study on two key modules: the *PassageCostEvaluator* and the *NavigationEventMonitor*. In the *PassageCostEvaluator* ablation, we removed this module, leaving only the *NavigationEventMonitor* to track events, allowing the system to avoid areas without relying on past experience. In the *NavigationEventMonitor* ablation, we excluded this module, enabling the system to use historical data but without current event tracking, to assess its importance in overall performance.

E. Multi-floor

We also tested our system’s ability to plan across floors, specifically avoiding obstacles like an elevator under maintenance. This test challenged the system to account for the robot’s limitations, such as avoiding the broken elevator and stairs, which are inaccessible for a wheeled robot, while still computing the shortest feasible path. Although a robot’s physical capabilities remain constant during navigation and could theoretically be hard-coded, we argue that it is essential for the LLM to understand the robot’s capabilities in order to make dynamic decisions, such as determining whether the robot can safely traverse small steps or slopes.

V. RESULTS

A. Effectiveness of NavigationEventMonitor

The *NavigationEventMonitor* demonstrated high accuracy in assessing the relevance of notifications to the robot’s navigation tasks. Out of 20 notifications (10 navigation relevant, 10 irrelevant), the LLM correctly identified whether each notification was relevant to robot navigation. Additionally, the model successfully identified and resolved all 20 outdated notifications by deleting out of date events.

The path validation results are summarized in Table I and evaluated by hand in terms of accuracy, recall, and false positive rate. Across 37 trials, 95 path approval requests were sent to ChatGPT-4o: 57 of 61 valid paths were approved (with `is_valid` set to `'true'` or the areas to avoid not in the path), and all 34 invalid paths were correctly flagged by setting `is_valid` to false. However, 14 requests out of 95, the responses were semantically correct but deviated from the specified output format, such as returning `'B sector'` or a room number instead of the exact area name, such that our navigation module could not parse them correctly. For DeepSeek-V3, 40 path approval requests were sent, evenly split between valid and invalid paths. All responses adhered strictly to the required output format. The findings highlight the high performance of both models in path assessment, confirming their practical utility in real-world applications.

TABLE II: Comparison of Average Path Lengths (m) Across Different Navigation Configurations for Various Cases

Navigation configurations	Case 1	Case 2	Case 3	Case 4
Our method	192.5	126.5	236.3	214.7
Our method w/o <i>NavigationEventMonitor</i>	221.2	135.4	264.3	223.3
Our method w/o <i>PassageCostEvaluator</i>	234.6	171.4	270.0	215.9
Move_base	347.3	202.2	398.5	267.5
Move_base w/o obstacle layer clearing	220.1	158.7	248.1	183.1

B. Comprehensive Comparison to Baseline Results

The results highlight significant differences in navigation performance between our method and baseline `move_base`. Our system, utilizing osmAG based navigation with external notifications and former experience, demonstrated superior ability to avoid restricted areas and adapt to the environment. In contrast, baseline `move_base` performance varied depending on the obstacle layer’s status. When the obstacle layer was cleared between trials, `move_base` often chose inefficient paths through closed doors and then re-planning. Without clearing, `move_base` improved in later trials, approximating the shortest path more closely. Regarding the interpretation of external information, `move_base` is of course performing very badly, because it does not have such capability, while our approach worked very well.

As shown in the inset figures in Fig. 5, our method outperformed `move_base` in the first trial of all four cases due to external information on restricted areas. In subsequent trials, our method, with stored experience, performed comparably to baseline `move_base` without obstacle layer clearing. Overall, our approach consistently outperformed `move_base` by maintaining context-aware path planning, demonstrating the effectiveness of integrating external information through *PassageCostEvaluator* and *NavigationEventMonitor*. Across 37 trials in 4 cases, our system consistently avoided restricted areas and successfully reached the designated destinations, reducing the total path length by 58% compared to baseline `move_base` with obstacle layer clearing between trials, as shown in Table II. Notably, in cases 2, 3, and 4 (Fig. 5 (b), (c), (d)), where not all the doors to restricted areas were closed, the baseline `move_base` entered restricted areas 25 times and failed 4 times across 74 runs in 37 trials, highlighting the clear advantage of our approach.

C. Ablation Experiment

As illustrated in Fig. 5 and Table II, the ablation studies provided insight into the contribution of individual components to the system’s overall performance.

1) *Ablation of PassageCostEvaluator*: When the *PassageCostEvaluator* is removed, the system still managed to avoid restricted areas but kept trying to pass infeasible passages over trials, leading to longer detours. This highlighted the importance of the *PassageCostEvaluator* in leveraging historical data to optimize navigation decisions.

2) *Ablation of NavigationEventMonitor*: Removing the *NavigationEventMonitor* significantly reduced the system’s ability to avoid restricted areas. In test cases where doors were closed in restricted areas, the robot eventually collected enough experience to perform similarly to our full method, though the initial few trials resulted in longer paths. However,

in tests where doors remained open in restricted areas, the system without this module violated the restrictions introduced by the notifications. This experiment demonstrates the importance of the *NavigationEventMonitor* in ensuring compliance with external information during navigation.

D. Multi-floor Navigation

In the multi-floor navigation test, our system effectively planned a path across different floors, particularly by avoiding areas like elevators under maintenance and stairs that the wheeled robot cannot climb. The *osmAGPathPlanner* proposed a path through the stairs, but the *NavigationEventMonitor* rejected it, based on our prompt that this is a wheeled robot and by looking up in the osmAG map, that this particular area is a stair. The accompanying video illustrates this capability, highlighting a distinct advantage over *move_base*, which lacks support for multi-floor navigation and does not consider the robot's capabilities.

VI. CONCLUSION AND LIMITATION

This paper presents the integration of Large Language Models into a robot path planning and navigation system, combining the efficiency and speed of the A^* algorithm with the broad knowledge and reasoning capabilities of LLMs. This hybrid approach leverages the strengths of both methods: A^* for precise, optimized pathfinding, and LLMs for understanding and adapting to human instructions and societal norms, utilizing our osmAG map representation, which is very suitable for LLMs.

Although this study was conducted using a map limited to our campus building and its associated notifications, we have demonstrated that current LLMs, possess the capability to comprehend textual maps and integrate real-world societal information. In the future, as the system is tested in more diverse real-world environments, we aim to collect and incorporate more detailed human preferences and contextual data. This could include avoiding crowded areas during peak hours or steering clear of sensitive locations like VIP conference rooms. This study lays a promising foundation for the continued integration of LLMs with traditional algorithms into everyday applications, paving the way for more intelligent and socially adaptive robotic systems in the future.

REFERENCES

- [1] J. Lin, H. Gao, X. Feng, R. Xu, C. Wang, M. Zhang, L. Guo, and S. Xu, "Advances in embodied navigation using large language models: A survey," *arXiv preprint arXiv:2311.00530*, 2023.
- [2] B. Yu, H. Kasaei, and M. Cao, "L3mvn: Leveraging large language models for visual target navigation," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 3554–3560.
- [3] G. Zhou, Y. Hong, and Q. Wu, "Navgpt: Explicit reasoning in vision-and-language navigation with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 7, 2024, pp. 7641–7649.
- [4] S. Y. Gadre, M. Wortsman, G. Ilharco, L. Schmidt, and S. Song, "Cows on pasture: Baselines and benchmarks for language-driven zero-shot object navigation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 23 171–23 181.
- [5] D. Feng, C. Li, Y. Zhang, C. Yu, and S. Schwertfeger, "Osmag: Hierarchical semantic topometric area graph maps in the osm format for mobile robotics," in *2023 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2023, pp. 1–7.
- [6] S. S. Fujing Xie, "Empowering robot path planning with large language models: osmag map topology & hierarchy comprehension with llms," in *2024 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2024, pp. 707–712.
- [7] F. Xie and S. Schwertfeger, "Robust lifelong indoor lidar localization using the area graph," *IEEE Robotics and Automation Letters*, vol. 9, no. 1, pp. 531–538, 2023.
- [8] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu *et al.*, "Palm-e: An embodied multimodal language model," *arXiv preprint arXiv:2303.03378*, 2023.
- [9] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choremanski, T. Ding, D. Driess, A. Dubey, C. Finn *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.
- [10] Y. Ding, X. Zhang, C. Paxton, and S. Zhang, "Task and motion planning with large language models for object rearrangement," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 2086–2092.
- [11] P. Sharma, B. Sundaralingam, V. Blukis, C. Paxton, T. Hermans, A. Torralba, J. Andreas, and D. Fox, "Correcting robot plans with natural language feedback," *arXiv preprint arXiv:2204.05186*, 2022.
- [12] A. Buckner, L. Figueredo, S. Haddadin, A. Kapoor, S. Ma, S. Vemprala, and R. Bonatti, "Latte: Language trajectory transformer," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 7287–7294.
- [13] X. Wang, W. Wang, J. Shao, and Y. Yang, "Lana: A language-capable navigator for instruction following and generation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 19 048–19 058.
- [14] S. S. Kannan, V. L. Venkatesh, and B.-C. Min, "Smart-llm: Smart multi-agent robot task planning using large language models," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 12 140–12 147.
- [15] S. H. Vemprala, R. Bonatti, A. Buckner, and A. Kapoor, "Chatgpt for robotics: Design principles and model abilities," *IEEE Access*, 2024.
- [16] D. Shah, B. Osifski, S. Levine *et al.*, "Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action," in *Conference on Robot Learning*. PMLR, 2023, pp. 492–504.
- [17] S. Chen, P.-L. Guhur, M. Tapaswi, C. Schmid, and I. Laptev, "Think global, act local: Dual-scale graph transformer for vision-and-language navigation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 16 537–16 547.
- [18] C. Huang, O. Mees, A. Zeng, and W. Burgard, "Visual language maps for robot navigation," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 10 608–10 615.
- [19] N. Yokoyama, S. Ha, D. Batra, J. Wang, and B. Bucher, "Vlfm: Vision-language frontier maps for zero-shot semantic navigation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 42–48.
- [20] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, "3d scene graph: A structure for unified semantics, 3d space, and camera," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 5664–5673.
- [21] N. Hughes, Y. Chang, and L. Carlone, "Hydra: A real-time spatial perception system for 3d scene graph construction and optimization," *arXiv preprint arXiv:2201.13360*, 2022.
- [22] J. Chen, D. Barath, I. Armeni, M. Pollefeys, and H. Blum, "“where am i?” scene retrieval with language," in *European Conference on Computer Vision*. Springer, 2024, pp. 201–220.
- [23] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Sunderhauf, "Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning," in *7th Annual Conference on Robot Learning*, 2023.
- [24] Z. Dai, A. Asgharivaskasi, T. Duong, S. Lin, M.-E. Tzes, G. Pappas, and N. Atanasov, "Optimal scene graph planning with large language model guidance," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 14 062–14 069.
- [25] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa *et al.*, "Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 5021–5028.
- [26] M. A. Eitan Marder-Eppstein, Vijay Pradeep, "Ros documentation on actionlib package," accessed: 2024-09-09. [Online]. Available: <https://wiki.ros.org/actionlib>